

PSS Assignment 2 2025 Group 16 - DevOps

Repository URL: [GitLab Repo](#)

Group Members:

- John Eric Arriola Mtr. 879341
- Nicolas Chines Mtr. 899536
- Marco Piccinini Marchini Mtr. 859316

Descrizione degli Stage

```
stages:  
  - build  
  - verify  
  - test  
  - package  
  - release  
  - docs  
  - deploy
```

Build Stage

```
build-backend:  
  stage: build  
  image: node:20.11.1-alpine  
  script:  
    - cd ./blogList-backend/  
    - npm install  
  artifacts:  
    paths:  
      - ./blogList-backend/node_modules/  
    expire_in: 1 hour  
  
build-frontend:  
  stage: build  
  image: node:20.11.1-alpine  
  script:  
    - cd ./bloglist-frontend/  
    - npm install  
  artifacts:  
    paths:  
      - ./bloglist-frontend/node_modules/  
    expire_in: 1 hour
```

In questo Stage vengono avviati 2 Jobs, build-frontend e build-backend, che installano le dipendenze utilizzate nei 2 subfolder.

I due Jobs creano gli artefatti node_modules dei due subfolder, che sono disponibili all'utilizzo dei Jobs successivi per 1 ora.

Verify Stage

```
verify-backend-eslint:
  stage: verify
  image: node:20.11.1-alpine
  script:
    - cd ./blogList-backend/
    - npm run lint
  dependencies:
    - build-backend

verify-frontend-eslint:
  stage: verify
  image: node:20.11.1-alpine
  script:
    - cd ./bloglist-frontend
    - npm run lint
  dependencies:
    - build-frontend
```

In questo Stage vengono avviati 2 Jobs, verify-backend-eslint e verify-frontend-eslint, che effettuano l'analisi dinamica del codice dei due subfolder tramite ESLint, controllando che seguano le linee guida sulla scrittura di codice dell'organizzazione (definiti nei file .eslint.cjs contenuti nei due subfolder).

Questo Stage utilizza gli artefatti generati durante lo Stage di Build.

Test Stage

```
integration-test:
  stage: test
  image: node:20.11.1-alpine
  script:
    - npm run test
  dependencies:
    - build-backend
```

In questo Stage viene effettuato l'Integration Test definito nel subfolder `./blogList-backend/test/`, viene chiamato lo script `test` definito nel `package.json` della root directory:

```
# package.json
```

```
"test": "cd ./blogList-backend/ && npm run test"
```

che chiama lo script `test` definito nel `package.json` contenuto nel subfolder `blogList-backend`:

```
# blogList-backend/package.json

"test": "cross-env NODE_ENV=test node --test --test-concurrency=1"
```

Imposta l'ambiente dell'applicazione in una modalità test, connettendosi a un database di test, in modo da non compromettere il database di produzione.

Effettua dei test sul corretto funzionamento degli endpoint dell'applicazione.

L'accesso al database viene effettuato tramite codice, connettendosi con MongoDB Atlas, ed è per questo che non è stato avviato un servizio MongoDB interno all'immagine della pipeline:

```
app.js

mongoose.connect(config.MONGODB_URI)
  .then(() => {
    logger.info('connected to MongoDB')
  })
  .catch((error) => {
    logger.error('error connecting to MongoDB: ', error.message)
  })
```

Questo Stage utilizza gli artefatti generati durante lo Stage di Build.

Package Stage

```
package:
  stage: package
  image: node:20.11.1-alpine
  script:
    - npm run build
  artifacts:
    paths:
      - ./build
    expire_in: 1 hour
  dependencies:
    - build-frontend
```

In questo stage, viene avviato un singolo Job che genera il folder di distribuzione del frontend `./dist`, lo inserisce nel subfolder `./blogList-backend`, e genera il folder di distribuzione dell'applicazione `./build`.

Viene chiamato lo script **build** definito nel **package.json** della root directory:

```
package.json
```

```
"build:frontend": "node ./scripts/buildFrontend.js",  
"build": "npm run build:frontend && node ./scripts/build.js"
```

Dove vengono avviati gli script di build contenuti nel subfolder **./scripts**

Questo Stage genera l'artefatto **./build** disponibile per 1 ora, che verrà poi utilizzato durante lo Stage di **Release**.

Questo Stage utilizza gli artefatti generati durante lo Stage di Build.